

Yazılım Testi: Oracle Problemi

ÖFY

August 2026

Özet

Bir testin “geçti” ya da “kaldı” kararını neye dayanarak verdiği, testin kendisinden bağımsız bir problemdir ve literatürde oracle problemi adıyla anılır. Yazılımın büyük bir kısmında beklenen çıktı önceden yazılamaz; bu durumda doğruluk, mutlak bir değerle değil, iki çalıştırma arasındaki ilişkiyle denetlenir. Metamorfik test bu ailenin en yerleşik üyesidir ve terminolojisini bütünüyle matematikten devralmıştır; bu devralma, yöntemin sektörde az tanınmasının başlıca sebebidir. Metamorfik ilişkinin sağlanması doğruluğu garanti etmez: aynı hatalı sonucu üreten iki çalıştırma arasında ilişki korunur, test geçer. Birim testi ile metamorfik testin birleşimi hata oranını ciddi biçimde düşürür ancak hatasızlık üretmez, çünkü en pahalı hatalar kodda değil gereksinimde ortaya çıkar. Test edilecek senaryonun akla gelmesi bir hayal gücü problemi olduğundan, çözümü daha dikkatli düşünmek değil, senaryo üretimini insandan alan yöntemlerdir. Havacılık ve savunma gibi hata maliyeti yüksek alanlar da hatayı sıfırlamaz; birbirinin kör noktasını kapatan katmanlar kurarak olasılığını düşürür.

Giriş

Yazılım testi iki ayrı işten oluşur. Birincisi programın hangi girdiyle çalıştırılacağına karar vermektir. İkincisi, çalıştırma bittikten sonra sonucun doğru olup olmadığına karar vermektir. Bu ikinci işe **test oracle** adı verilir. Kelime, doğruyu bildiğine inanılan kâhin anlamındaki *oracle* sözcüğünden gelir ve testin doğruluk kaynağını ifade eder.

Bir toplama fonksiyonunun testinde beklenen değerin 5 olarak yazılması, en yalın oracle biçimidir. Doğru cevap bilinmekte ve test ona bakmaktadır. Ancak gerçek sistemlerin çoğunda doğru cevap önceden yazılamaz. Beş bin satırlık veriden rapor üreten bir fonksiyonun çıktısı elle hesaplanamaz; çok katmanlı bir yetki kuralının belirli bir kullanıcı için sonucu önceden bilinemez; bir hava durumu modelinin çıktısı zaten bilinseydi modele ihtiyaç olmazdı. Doğru cevabın erişilemez olduğu bu duruma **oracle problemi** denir ve terim ilk kez Weyuker (1982) tarafından “test edilemeyen programlar” başlığı altında formüle edilmiştir.

Oracle problemi, testin bir yan meselesi değil merkezî kısıttır. Girdi üretimi otomatikleştirilebilir, kapsam ölçülebilir, ancak sonucun doğruluğuna karar verme işi otomatikleştirilmesi en zor bileşendir. Barr ve arkadaşlarının (2015) alan taraması, oracle otomasyonundaki eksikliği test otomasyonunun asıl darboğazı olarak tanımlar.

Bu notta kullanılan üç terim baştan ayrılmalıdır. **Doğrulama** (*verification*), yazılımın kendi spesifikasyonuna uyup uymadığının denetimidir. **Geçerleme** (*validation*), spesifikasyonun kendisinin doğru ihtiyaca karşılık gelip gelmediğinin denetimidir. Ayrım Boehm (1979) tarafından iki soruyla özetlenmiştir: ürünü doğru mu yapıyoruz, doğru ürünü mü yapıyoruz. **Test**, doğrulamanın yalnızca bir yöntemidir; statik analiz, biçimsel doğrulama ve kod incelemesi diğer yöntemlerdir.

Notun kapsamı, doğruluk kararının nasıl verildiği ve bu kararın sınırlarıdır. Test girdisinin nasıl seçileceği ve kapsam ölçütlerinin nasıl tanımlanacağı yalnızca oracle ile ilişkili olduğu ölçüde ele alınır.

Oracle türleri

Oracle'lar, doğruyu nereden türettiklerine göre ayrılır. Aşağıdaki sınıflandırma Barr ve arkadaşlarının (2015) taramasında kullanılan ayrımı izler.

Spesifik oracle

Beklenen çıktının doğrudan yazıldığı biçimdir. Test, hesaplanan değeri elle belirlenmiş bir değerle karşılaştırır. Gündelik test yazımının neredeyse tamamı bu türdendir ve terimin varlığının fark edilmemesinin sebebi budur: oracle her zaman aynı ve aşikâr olduğunda, ayrı bir kavram olarak adlandırılmasına ihtiyaç duyulmaz.

Sınırı, doğru cevabın önceden bilinmesi şartıdır. Bu şart sağlanamadığında tür kullanılamaz.

Türetilmiş oracle

Doğru cevabın bilinmediği, ancak sonucun sağlanması gereken bir ilişkinin bilindiği durumlarda kullanılır. Doğruluk, mutlak bir değerden değil, başka bir çalıştırmadan, başka bir uygulamadan ya da ters bir işlemde türetilir. Beş yerleşik biçimi vardır.

Metamorfik oracle

Girdi denetimli biçimde değiştirilir ve çıktının nasıl değişmesi gerektiği kontrol edilir. Ayrıntısı izleyen bölümde.

Regresyon oracle

Önceki sürümün çıktısı bir dosyaya kaydedilerek doğru kabul edilir; sonraki çalıştırmalar bu kayıtlarla karşılaştırılır. Kayda *golden file* ya da *snapshot* adı verilir. Doğruluk iddiası zayıftır: kontrol edilen şey doğruluk değil, değişmezliktir. Kaydedilen çıktı baştan hatalıysa hata kalıcı olarak doğru sayılır.

Diferansiyel oracle

Aynı işi yapan iki bağımsız uygulama aynı girdiyle çalıştırılır ve çıktılar karşılaştırılır. Fark çıkması en az birinin hatalı olduğunu gösterir, ancak hangisinin hatalı olduğunu göstermez. Aynı standardı uygulayan birden fazla yazılımın bulunduğu alanlarda kullanılır: derleyiciler, ayrıştırıcılar, protokol uygulamaları, veritabanı motorları.

Tersine çevirme oracle

Bir işlem uygulanır, ardından tersi uygulanır ve başlangıç durumuna dönülüp dönülmediğine bakılır. Serileştirme ile geri okuma, şifreleme ile çözme, sıkıştırma ile açma bu kalıba girer. Doğru çıktının ne olduğunu bilmeye gerek bırakmaz; yalnızca döngünün kapandığı denetlenir.

Model tabanlı oracle

Sistemin basitleştirilmiş bir modeli yazılır. Model yavaştır ancak doğruluğuna güvenilir; asıl sistem hızlıdır ancak doğruluğu denetlenmek istenmektedir. İkisi aynı girdiyle çalıştırılıp sonuçları karşılaştırılır.

İma edilen oracle

Hiçbir yerde yazılmasa da her programın sağlaması beklenen şartlardır: çökmek, kilitlenmemek, bellek ihlali üretmemek, sonsuz döngüye girmemek. Beklenen çıktı hakkında hiçbir bilgi gerektirmediği için, rastgele bozuk girdi üreten yöntemlerin dayandığı oracle budur.

İnsan oracle

Kararı bir insanın verdiği durumdur. Arayüzün doğru görüldüğü, bir metnin anlaşılır olduğu, bir sesin bozulmadığı otomatik olarak denetlenemediğinde başvurulur. Pahalıdır, tekrarlanabilirliği düşüktür ve ölçeklenmez.

Kısmi oracle

Çıktının tamamının değil, sağlaması gereken şartların denetlendiği biçimdir. Bir sıralama fonksiyonunun hangi sonucu üreteceği bilinmese de, çıktının sıralı olması ve eleman sayısının değişmemesi gerektiği bilinir. Ön koşul, son koşul ve değişmez (*invariant*) üçlüsüyle kurulan sözleşme yaklaşımı bu kümededir.

Metamorfik test

Tanım ve temel kavramlar

Metamorfik test, girdiyi denetimli biçimde dönüştürüp çıktının nasıl değişmesi gerektiğini denetleyen yöntemdir. Yöntem Chen ve arkadaşları tarafından 1998’de ortaya konmuştur.

Merkezî kavram **metamorfik ilişki**dir (*metamorphic relation*): iki çalıştırma arasında sağlanması gereken kural. İlk çalıştırmaya **kaynak test durumu**, dönüştürülmüş girdiyle yapılan ikinci çalıştırmaya **takip test durumu** adı verilir. Kaynağı takip durumuna çeviren işlem **dönüşüm fonksiyonu**, iki çıktı arasında beklenen bağ **çıktı ilişkisi**, ilişkinin bozulması ise **ihlaltir**. Test başarısızlığı bu ihlal demektir.

Somut bir örnek: çok kullanıcı bir sistemde belirli bir işlemin sonucunun ne olacağı önceden yazılamıyorsa, işlem önce bir kullanıcı için, ardından eşdeğer haklara sahip başka bir kullanıcı için çalıştırılır. Beklenen ilişki, iki sonucun aynı olmasıdır. Doğru sonucun ne olduğu bilinmemekte, ancak kullanıcı kimliğinin sonucu değiştirmemesi gerektiği bilinmektedir.

İlişki türleri

- **Eşitlik ilişkisi.** İki çıktı aynı olmalıdır. Yukarıdaki kullanıcı örneği bu türdendir.
- **Sıralama ilişkisi.** Girdi büyütüldüğünde çıktının belirli yönde değişmesi beklenir; örneğin bir listeye eleman eklendiğinde toplamın azalmaması.
- **Ölçekleme ilişkisi.** Girdinin sabit bir katsayıyla çarpılması durumunda çıktının nasıl değişeceği.
- **Bileşim ilişkisi.** İki işlemin ayrı ayrı uygulanmasıyla birlikte uygulanmasının aynı sonucu vermesi.
- **Kapsama ilişkisi.** Bir arama sorgusu daraltıldığında sonuç kümesinin büyümemesi.

Bu ilişkilerin bulunmasına **ilişki keşfi** denir ve yöntemin en zor kısmıdır; ilişkiyi üreten otomatik bir mekanizma yoktur, alan bilgisi gerektirir.

Terminolojinin matematiksel kökeni

Metamorfik ilişkilerin adları doğrudan matematikten alınmıştır ve bu, yöntemin sektörde yerleşmemesinin başlıca sebebidir.

- **Simetri.** Bir dönüşüm uygulandığında nesnenin değişmemesi. Yazılımdaki karşılığı: işlemi kimin yaptığının sonucu değiştirmemesi.
- **Değişmezlik** (*invariance*). Bir şey değişirken başka bir şeyin sabit kalması. Simetrinin daha genel adıdır.

- **Etkisizlik** (*idempotency*). Bir işlemin iki kez uygulanmasının bir kez uygulanmasıyla aynı sonucu vermesi.
- **Permütasyon**. Sıranın değişmesinin sonucu değiştirmemesi.
- **Ters işlem**. Bir işlemin tersinin uygulanmasıyla başlangıca dönülmesi.

Gündelik konuşmada bu ilişkilerin sırasıyla “kim yaparsa yapsın fark etmemeli”, “iki kere basınca iki kere olmamalı”, “sıra önemli olmamalı” ve “geri alınca başa dönmeli” biçiminde ifade edildiği görülür. İki dil arasındaki mesafe, kavramın ekip içinde bilinmesine karşın literatürdeki adıyla tanınmamasına yol açar.

Uygulama alanları

Yöntem, doğru çıktının bilinemediği alanlarda yoğunlaşır. Bilimsel hesaplama ve simülasyon yazılımları, biyoinformatik araçları bunların başında gelir. Makine öğrenmesi modellerinin testinde de yaygınlaşmıştır: bir görüntüye küçük bir dönüşüm uygulandığında sınıflandırma sonucunun değişmemesi gerektiği, metamorfik bir ilişkidir.

En görünür başarısı derleyici testindedir. Csmith aracı rastgele geçerli program üreterek farklı derleyicilerin sonuçlarını karşılaştırmış (Yang ve ark., 2011), EMI yaklaşımı ise programı davranışını değiştirmeyecek biçimde dönüştürüp derlenmiş hâlin aynı sonucu vermesi gerektiği ilişkisine dayanmıştır (Le, Afshari ve Su, 2014). Her iki çalışma da yaygın kullanılan derleyicilerde yüzlerce hata ortaya çıkarmıştır.

Yöntemin kör noktası

Metamorfik ilişkinin sağlanması doğruluğu göstermez. İki çalıştırma da aynı hatalı sonucu üretiyorsa ilişki korunur ve test geçer. Simetri korunmuş, doğruluk korunmamıştır. Bu, yöntemin yapısal sınırını ve başka oracle türleriyle birlikte kullanılmasını gerektirir.

İkinci sınır hata ayıklamadadır. İlişki ihlal edildiğinde iki çalıştırmadan hangisinin hatalı olduğu bilinmez; ihlal yalnızca en az birinin yanlış olduğunu söyler.

Testin yerleřtiđi kalite çerçevesi

Test, tek başına duran bir faaliyet deđildir. Üstünde birbirinden ayrı dört başlık bulunur ve bunlar pratikte sık karışırılır.

Kalite kontrol

Üretilen ürünün muayenesidir. Test etmek, kod incelemesi, hata sayımı, kabul testi bu başlığa girer. Sorusu şudur: bu ürün doğru mu.

Alt terimleri arasında **kabul kriteri** (bir işin bitmiş sayılması için sağlanması gereken şartlar), **regresyon** (önceden çalışın bir işlevin yeni deđişiklikle bozulması), **yeniden test** (düzeltildiđi bildirilen hatanın gerçekten düzelip düzelmediđinin denetimi) ve **kaçak oranı** (test aşamasında yakalanamayıp kullanıcıya ulaşın hataların oranı) bulunur. Son ölçüt, test setinin kalitesini deđerlendirmenin doğrudan yollarından biridir.

Hata terimleri arasındaki ayrım burada kurulur: *fault* koddaki yanlışın kendisi, *failure* kullanıcının gördüğü bozukluk, *defect* ise kayda geçmiş sorundur. Hiç çalıştırılmayan bir fault failure'a dönüşmez.

Kalite güvence

Ürünle deđil süreçle ilgilenir. Sorusu şudur: doğru biçimde mi çalışıyoruz. Her deđişikliđin en az bir kiři tarafından onaylanması kuralı, sürüm çıkarma adımlarının yazılı olması, belirli saatlerden sonra dağıtım yapılmaması kararı bu başlığa girer.

Merkezî terimleri **süreç** (bir işin izlediđi tanımlı yol), **standart** (herkesin uyduđu yazım ve adlandırma kuralı), **prosedür** (belirli bir işin adım adım yazılı tarifi), **politika** (bağlayıcı karar) ve **önleyici faaliyet**dir. Sonuncusu ayrımın özünü taşır: hatayı çıktıktan sonra düzeltmek kalite kontrol, çıkmasını baştan engelleyecek düzenlemeyi kurmak kalite güvencedir.

Denetim ve uygunluk

Dışarıdan gelen kurallara uygunluğun denetimi ve kanıtlanmasıdır. **Denetim izi**, sistemde kimin ne zaman ne yaptıđının geriye dönük kaydıdır. **Kanıt**, bir uygulamanın yapıldıđını gösterebilme yükümlülüğüdür; kanıtı olmayan uygulama denetimde yapılmamış sayılır. **Uygunsuzluk** kuralla gerçeğin ayrıştığı durum, **bulgu** denetçinin raporuna geçirdiđi tespit, **düzeltici faaliyet** ise bu tespite karşılık tarih ve sorumluyla tanımlanmış somut düzeltmedir.

İzlenebilirlik, bir gereksinimden tasarıma, koda ve teste uzanan zincirin kurulabilmesidir. İki yönlü çalışır: hangi kodun hangi gereksinim için var olduđu ve hangi gereksinimin hangi testle karşılandıđı sorulabilir.

Süreç iyileştirme

Kuralın işe yarayıp yaramadığına bakıp kuralı değiştirme faaliyetidir. **Kök neden analizi**, sorunun görünen sebebini değil kaynağını arama yöntemidir. **Metrik**, iyileşmenin ölçülmesini sağlayan sayıdır. **Retrospektif**, ekibin dönem sonunda neyin işlediğini ve neyi değiştireceğini konuşmasıdır; çıktısı somut bir değişiklik olmadığında işlevini yitirir. **Suçlamasız inceleme** (*blameless post-mortem*), arıza sonrası kişiyi değil sistemi inceleme yaklaşımıdır; suçlamanın bulunduğu ortamda bilgi saklanır ve öğrenme durur.

Üç faaliyetin ayrımı

Ayrım tek cümlede kurulabilir: kalite kontrol hatayı yakalar, kalite güvence hatanın oluşmasını engelleyecek kuralı koyar, süreç iyileştirme kuralın etkisini ölçüp kuralı değiştirir.

Uygulamada bu ayrım sık bulanıklaşır. Yazılım şirketlerinde “kalite güvence ekibi” adıyla anılan grupların büyük bölümü kalite kontrol işi yapar; test eder, süreç tasarlamaz.

Testin sınırları

Yapısal sınır

Testin temel kısıtı Dijkstra (1970) tarafından formüle edilmiştir: test, hataların varlığını göstermeye yarar, yokluğunu göstermeye yaramaz. Çalıştırılan test sayısı sonludur, olası girdi uzayı ise pratikte sonsuzdur. Metamorfik test tek bir testin kapsadığı alanı genişletir ancak bu boşluğu kapatmaz.

Bileşim sınırı

Parçaların ayrı ayrı doğruluğu bütünün doğruluğunu vermez. Tüm birim testleri geçen bir sistemin bileşenler arası sınırlarda bozulması sık görülen durumdur. Çağrı sıralarının yanlışlığı, veri sözleşmelerinin uyumsuzluğu, eşzamanlılıktan doğan yarış koşulları, zaman aşımı ve yeniden deneme davranışları birim testinde görünmez, çünkü bu testler tanımı gereği bileşeni yalıtarak çalıştırır.

Ortam sınırı

Yük altındaki davranış, ağ kesintisinde ortaya çıkan durum, veritabanı seviyesinde kilitlenme, yarıda kalan veri göçü gibi olgular test tipiyle değil ortamla ilgilidir. Üretim ortamının test ortamından farklı olması, testte görünmeyen hataların en yaygın kaynaklarından biridir.

Gereksinim sınırı

En pahalı hatalar kodda değil gereksinimde ortaya çıkar. Kodun spesifikasyona uyması, spesifikasyonun doğru olduğu anlamına gelmez. Doğrulama başarılı, geçiştirme başarısız olabilir; kusursuz çalışan yanlış özellik bu durumun adıdır.

İlişki keşfi sınırı

Metamorfik testte denetlenen ilişkiler, akla gelmiş olan ilişkilerdir. Akla gelmeyen ilişki test edilmez ve hangi ilişkilerin eksik kaldığını ölçen bir mekanizma yoktur. Kapsam ölçütleri kodun ne kadarının çalıştırıldığını gösterir, hangi özelliklerin denetlenmediğini göstermez.

Keşif problemi ve çözüm yolları

Test edilecek senaryonun akla gelmesi bir hayal gücü problemidir. Çözümü daha dikkatli düşünmek değil, senaryo üretimini insandan alan yöntemlerdir.

Rastgele girdi üretimi

Özellik tabanlı test (*property-based testing*) yaklaşımında geliştirici tek tek girdiler değil, her girdide sağlanması gereken bir özellik yazar. Girdileri araç üretir ve genellikle binlerce kombinasyon dener. Hata bulunduğunda araç, hatayı üreten girdiyi en küçük hâline indirger; bu indirgeme, nedenin görünür hâle gelmesini sağlar. Yaklaşım Claessen ve Hughes'ın (2000) çalışmasıyla yaygınlaşmıştır.

Bu yöntem metamorfik testle doğal olarak birleşir: yazılan özellik çoğunlukla bir metamorfik ilişkidir ve girdi üretimi elle seçilmiş birkaç durumdan çok daha geniş bir alanı tarar.

Bozuk girdi taraması

Bozuk girdi taraması (*fuzzing*), beklenen çıktıya dair hiçbir bilgi kullanmadan aşırı, bozuk ve anlamsız girdi üreterek çökme, kilitlenme ve bellek ihlali arar. Dayandığı oracle ima edilen türdendir. Dışarıdan veri alan her sınır, özellikle ayrıştırıcılar bu yöntemin doğal hedefidir.

Mutasyon testi

Mutasyon testi, koda kasten hata enjekte eder ve testlerin bu hatayı yakalayıp yakalamadığını ölçer. Bir karşılaştırma tersine çevrilir, bir koşul kaldırılır, bir sabit değiştirilir; testler yine de geçiyorsa o testler o bölgeyi denetlemiyor demektir.

Bu yöntemin önemi şuradadır: kapsam ölçütü kodun çalıştırıldığını gösterir, denetlendiğini göstermez. Hiçbir doğrulama içermeyen bir test de kapsamı artırır. Testlerin gerçekten hata yakalayıp yakalamadığını ölçen doğrudan yöntem mutasyon testidir.

Gerçek hatalardan geri besleme

Üretimde ortaya çıkan her hata için “bu neden testlerde görünmedi” sorusunun sorulması ve cevabın bir teste dönüştürülmesi. Hayal gücüne değil gerçekleşmiş olaya dayandığı için en verimli kaynaktır; zamanla test seti, varsayılan hata dağılımına göre değil gerçek hata dağılımına göre biçimlenir.

Testin ötesindeki katmanlar

Testin kapsamadığı alan, çalışma zamanındaki denetimlerle daraltılır.

- **Çalışma zamanı değişmez denetimi.** Testte denetlenen ilişkinin üretim ortamında da denetlenmesi. Test yalnızca test ortamında ve seçilmiş

girdilerle çalışır; değişmez denetimi her gerçek istekte çalışır ve ihlali kaydeder.

- **Kademeli yayın.** Yeni sürümün önce küçük bir kullanıcı kesimine verilmesi. Hata varsa etki alanı sınırlı kalır.
- **Gözlemlenebilirlik.** İzleme, bir ölçütün eşiği aştığını bildirir; gözlemlenebilirlik, hangi istekte, hangi kullanıcıda ve hangi adımda sorunun çıktığını sorgulanabilir kılar. Kayıt, ölçüt ve izleme verilerinin birlikte tutulmasıyla sağlanır.
- **Toparlanma süresi.** Hatanın hiç olmaması değil, ortaya çıktığında hızla geri alınabilmesi. Geri alma işlemi kısa sürüyorsa hatanın maliyeti düşer.

İzleme tek başına bir çözüm değildir, çünkü hatayı ancak kullanıcı onunla karşılaştıktan sonra görür. Savunmanın son katmanıdır, birincisi değil.

Hata maliyeti yüksek sistemler

Bu alanlarda da hata olur

Havacılık ve savunma yazılımlarının hatasız olduğu varsayımı yanlıştır. Ariane 5 roketinin ilk uçuşu, 64 bitlik bir kayan noktalı değerin 16 bitlik tam sayıya dönüştürülmesi sırasında oluşan taşma nedeniyle kalkıştan kırk saniye sonra kaybedilmiştir; hatalı modül o uçuşta işlevsel bir görev üstlenmiyordu ve önceki modelden değiştirilmeden devralınmıştı (Lions, 1996). Mars Climate Orbiter, iki ekibin farklı ölçü birimleri kullanması nedeniyle kaybedilmiştir (NASA, 1999). Boeing 737 MAX'te uçuş kontrol yazılımının tek bir sensöre dayanması, iki uçağın düşmesiyle sonuçlanan olaylar zincirinde belirleyici olmuştur.

F-35 programının kendi geçmişinde de radar yazılımının uçuş sırasında yeniden başlatılmasını gerektiren kararlılık sorunları, navigasyon hataları ve lojistik yazılımının uçakları hatalı biçimde arızalı göstermesi gibi olaylar kayıtlıdır.

Bu örneklerin ortak yanı dikkat çekicidir: kodun büyük bölümü kendi spesifikasyonuna uygundu. Ariane'da hata, eski roket için geçerli olan bir varsayımın yeni roket için hâlâ geçerli sayılmasındaydı. Yani en pahalı hatalar doğrulama değil geçişleme hatasıdır.

Uygulanan yöntemler

Biçimsel doğrulama

Kodun doğruluğunun test edilerek değil matematiksel olarak kanıtlanması. Önce sistemin ne yapması gerektiği matematiksel bir dille yazılır; buna biçimsel spesifikasyon denir. Ardından kodun bu spesifikasyonu sağladığı araçlarla kanıtlanır. Başlıca teknikleri teorem ispatlama, model denetimi (sistemin ulaşabileceği tüm durumların taranıp istenmeyen bir duruma erişilip erişilemediğine bakılması) ve soyut yorumlamadır.

Yöntemin sınırı, kanıtın spesifikasyonun doğruluğunu göstermemesidir; yanlış bir spesifikasyon kusursuz biçimde kanıtlanabilir. Maliyeti yüksek olduğu için genellikle yalnızca en kritik çekirdek modüllere uygulanır. Havacılık dışında da kullanılmıştır: Amazon, dağıtık sistemlerinin tasarım hatalarını bulmak için biçimsel spesifikasyon kullandığını bildirmiştir (Newcombe ve ark., 2015).

Yedeklilik ve oylama

Aynı işi yapan birden fazla bağımsız birimin çalıştırılıp sonuçlarının karşılaştırılması. Üç birimli düzenekte ikisinin uyduğu sonuç kabul edilir, ayrışan birim devre dışı bırakılır.

İki biçimi vardır. Donanım yedekliliği aynı yazılımı farklı bilgisayarlarda çalıştırır; donanım arızasına karşı korur ancak yazılım hatası tüm birimlerde aynı anda ortaya çıkar. Yazılım yedekliliğinde aynı gereksinim farklı ekiplerce, birbirinden bağımsız olarak yazılır. Bu yaklaşımın varsayımı, bağımsız ekiplerin aynı hatayı yapmayacağıdır. Knight ve Leveson (1986) bu varsayımı deneysel olarak sınamış ve bağımsız gerçekleştirmelerin hatalarının istatistiksel olarak

bağımsız olmadığını, farklı ekiplerin benzer zorluk noktalarında benzer hatalar ürettiğini göstermiştir. Yöntem hata korelasyonunu düşürür ancak ortadan kaldırmaz.

Donanım seviyesinde izolasyon

Kritik ve kritik olmayan yazılımın birbirini etkileyememesi. Uçuş kontrolü ile ikincil sistemler aynı kaynağı paylaşırsa, birindeki hata ya da aşırı kaynak tüketimi diğerini etkileyebilir. İzolasyon bunu ayrı işlemciler, bellek koruma birimleri, her göreve sabit zaman dilimi ve sabit bellek ayıran işletim sistemleri ve ayrılmış veri yollarıyla engeller. Bu yaklaşıma bölümlendirme (*partitioning*) denir.

Tam izlenebilirlik

Her gereksinimin hangi tasarıma, koda ve teste karşılık geldiğinin uçtan uca kayıtlı olması. İki işlevi vardır. Birincisi hiçbir gereksinimin karşılıksız kalmadığını gösterir. İkincisi, hiçbir gereksinime bağlanmayan kodun bulunmadığını gösterir; bu tür kod kimsenin talep etmediği ve dolayısıyla sistematik olarak test edilmemiş koddur ve bu alanlarda kabul edilmez.

Standartlara zorunlu uyum

Bu alanlarda yazılım geliştirme belgelenmiş süreçlere bağlıdır. Havacılıkta DO-178C, otomotivde ISO 26262, demiryolunda EN 50128, tıbbi cihazlarda IEC 62304 uygulanır. Standartlar yazılıma kritiklik seviyesi atar; arızası ölümle sonuçlanabilecek en üst seviyede, kaynak kodun her koşulunun bağımsız olarak denetlendiğini gösteren ağır kapsam şartları ve bağımsız gözden geçirme yükümlülüğü gelir. Uyum sertifikasyonla kanıtlanır ve belgelenmemiş faaliyet yapılmamış sayılır.

Simülasyon ve arıza enjeksiyonu

Gerçek kullanımdan önce sistemin kademeli biçimde sınanması: önce bütünüyle yazılımsal simülasyon, ardından gerçek kontrol biriminin simüle edilmiş sensörlere bağlanması, sonra tüm sistemin yer tesisinde birleştirilmesi.

Bu aşamalarda aranan şey normal davranış değil arıza davranışdır. Sensörün bozulması, iki sensörün çelişmesi, gücün kesilmesi, veri yolunun kopması gibi durumlar gerçekte denenemeyeceği için yapay olarak enjekte edilir. Buna arıza enjeksiyonu denir.

Katmanların ortak mantığı

Altı yöntemin hiçbirisi tek başına yeterli değildir. Kanıt spesifikasyonun doğruluğunu göstermez, yedeklilik ortak kökenli hataları yakalamaz, simülasyon her senaryoyu içermez, standarda uyum biçimsel kalabilir. Katmanlar üst üste konduğunda her birinin kör noktası bir diğeri tarafından kapatılır. Bir hatanın zarara dönüşmesi için tüm katmanların aynı noktada delinmesi gerekir.

Bu yaklaşımın bedeli, satır başına maliyetin sıradan yazılımın kat kat üzerinde olması ve geliştirme hızının belirgin biçimde düşmesidir. Hata olasılığını bir mertebe daha aşağı çekmek, maliyeti doğrusal değil üstel biçimde artırır.

Sonuç

Oracle kavramının ayrı bir isimle anılması, testin iki bağımsız kararı içerdiğini görünür kılar: neyin çalıştırılacağı ve sonucun doğru sayılıp sayılmayacağı. Bu ayrım yapıldığında, beklenen değerin yazılamadığı durumlarda testten vazgeçmek gerekmediği ortaya çıkar; doğruluk mutlak bir değerle değil, çalıştırmalar arası bir ilişkiyle de denetlenebilir.

Metamorfik ilişkilerin matematiksel adlarla anılması bir tercih değil, yöntemin doğrudan matematiksel bir çerçeveden türetilmiş olmasının sonucudur. Bu durum kavramın sektörde tanınmasını zorlaştırmıştır; ekipler ilişkiyi çoğunlukla gündelik dille ifade ederek zaten kullanmakta, ancak literatürdeki karşılığını bilmedikleri için yöntemin geri kalanından yararlanamamaktadır. Terimi bilmenin pratik değeri buradadır: ilişki türleri listesi, akla gelmemiş denetimleri hatırlatan bir kontrol listesi işlevi görür.

Yöntemlerin birleştirilmesi hatasızlık üretmez. Birim testi bileşenleri yalıtılarak denetler ve bileşim hatalarını göremez; metamorfik test ilişkiyi denetler ve iki tarafın birlikte yanılması durumunu göremez; her ikisi de gereksinimin yanlışlığına karşı kördür. Kazanç, hatanın ortadan kalkması değil olasılığının düşmesidir.

Bundan çıkan pratik sonuç, hedefin yer değiştirmesidir. Hatanın önlenmesi ulaşılamaz bir hedef olduğu için, olgun uygulama hatanın maliyetini düşürmeye yönelir: değişikliği küçük tutmak, etkiyi kademeli yaymak, hızlı fark etmek, hızlı geri almak ve ancak sonrasında kök nedene inmek. Hata maliyeti yüksek alanların yaptığı da esasen budur; oradaki fark, katman sayısının fazlalığı ve her katmanın belgelenmiş olmasıdır.

Notun cevaplamadığı iki soru açık kalmaktadır. Birincisi, metamorfik ilişkilerin keşfinin ne ölçüde sistematikleştirilebileceğidir; hâlihazırda bu iş alan bilgisine ve sezgiye bağlıdır ve eksik kalan ilişkileri ölçen bir ölçüt yoktur. İkincisi, çalışma zamanında denetlenen değişmezlerin maliyet ve fayda dengesidir; her istekte çalışan denetim performans etkiler, ancak bu etkinin hangi kritiklik seviyesinden itibaren kabul edilebilir olduğu genel bir kuralla belirlenmiş değildir.

Kaynaklar

- Barr, E. T., Harman, M., McMinn, P., Shahbaz, M., Yoo, S. (2015). The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering*, 41(5).
- Boehm, B. W. (1979). Guidelines for Verifying and Validating Software Requirements and Design Specifications. *Euro IFIP*.
- Chen, T. Y., Cheung, S. C., Yiu, S. M. (1998). Metamorphic Testing: A New Approach for Generating Next Test Cases. Teknik rapor, Hong Kong University of Science and Technology.
- Chen, T. Y., Kuo, F.-C., Liu, H., Poon, P.-L., Towey, D., Tse, T. H., Zhou, Z. Q. (2018). Metamorphic Testing: A Review of Challenges and Opportunities. *ACM Computing Surveys*, 51(1).

- Claessen, K., Hughes, J. (2000). QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. *ICFP*.
- Dijkstra, E. W. (1970). *Notes on Structured Programming*. Technische Hogeschool Eindhoven.
- Knight, J. C., Leveson, N. G. (1986). An Experimental Evaluation of the Assumption of Independence in Multiversion Programming. *IEEE Transactions on Software Engineering*, SE-12(1).
- Le, V., Afshari, M., Su, Z. (2014). Compiler Validation via Equivalence Modulo Inputs. *PLDI*.
- Lions, J. L. (1996). *ARIANE 5 Flight 501 Failure: Report by the Inquiry Board*. ESA/CNES.
- NASA (1999). *Mars Climate Orbiter Mishap Investigation Board Phase I Report*.
- Newcombe, C., Rath, T., Zhang, F., Munteanu, B., Brooker, M., Deardeuff, M. (2015). How Amazon Web Services Uses Formal Methods. *Communications of the ACM*, 58(4).
- RTCA (2011). *DO-178C: Software Considerations in Airborne Systems and Equipment Certification*.
- Weyuker, E. J. (1982). On Testing Non-testable Programs. *The Computer Journal*, 25(4).
- Yang, X., Chen, Y., Eide, E., Regehr, J. (2011). Finding and Understanding Bugs in C Compilers. *PLDI*.